System Design

Fundamentals Interview Notes

A practical guide to approaching scalable system design problems

Core flow: Requirements → Scale → Architecture → Bottlenecks → Trade-offs

1. What Is System Design?

System design is the process of deciding how a software system should be structured so it can meet its functional and non-functional requirements at the expected scale.

In a small application, one server and one database may be enough. As traffic, data, users, and reliability expectations grow, the design usually needs more pieces such as load balancers, caches, queues, replicas, partitions, monitoring, and failure of handling.

2. What Does System Design Cover?

- **Requirements:** What should the system do, and what is explicitly out of scope?
- **Scale:** How many users, requests, writes, reads, and how much data should it handle?
- **Data:** What data is stored, where is it stored, and how is it accessed?
- **APIs and communication:** How do clients and services talk to each other?
- **Reliability:** What happens when a server, database, network link, or external service fails?
- **Performance:** What latency and throughput targets matter most?
- **Consistency:** Which operations must be strongly correct, and where is eventual consistency acceptable?
- **Operations:** How will the system be monitored, debugged, deployed, and evolved?
- **Cost:** Is the design reasonable for the expected business and traffic scale?

3. Basic Building Blocks

Most system designs can be explained by combining a small set of reusable building blocks. The exact architecture depends on requirements, scale, consistency, latency, and failure expectations.



Figure 1. Basic system design building blocks

- **Client:** The web app, mobile app, browser, or service that sends requests.
- **API Layer:** The interface clients used to interact with the system. It can handle routing, authentication, validation, rate limiting, and request/response contracts.
- **Load Balancer:** Distributes traffic across multiple application servers and can improve availability by routing around unhealthy instances.
- **Application Servers:** Run business logic and coordinate reads, writes, validations, external calls, and workflows.
- **Cache:** Stores frequently accessed data so the system can respond faster and reduce database load. Common patterns include cache-aside, write-through, and write-back.
- **Database or Storage:** Persists on the system's durable data. The choice depends on access patterns, consistency, scale, and query requirements.
- **Message Queue or Event Log:** Decouples producers and consumers, especially for asynchronous work. It helps absorb traffic spikes and enables retryable background processing.
- **External Services:** Payment providers, email/SMS providers, identity systems, analytics tools, and other third-party dependencies.
- **Observability Stack:** Logs, metrics, traces, alerts, and dashboards used to understand production behavior and diagnose failures.

4. How to Approach a System Design Problem

Step 1: Clarify the Requirements

- What are the core features?
- Who are the users?
- What should we exclude?
- What scale are we designing for?
- Which operations need low latency?
- Which operations need strong correctness?
- What availability or durability expectations exist?

Step 2: Estimate the Scale

- Read and write requests per second (RPS/QPS).
- Storage growth and total data volume.
- Bandwidth requirements.
- Peak traffic patterns and traffic-to-average ratio.
- Hot keys, hot partitions, or other uneven access patterns.
- Approximate concurrency and request size.

Step 3: Start With a Simple Architecture

- Client
- API layer
- Application service
- Database

Step 4: Identify Bottlenecks and Failure Modes

- What happens if the database goes down?
- What happens if traffic suddenly increases 10×?
- What happens if the cache is empty?
- What happens if a queue consumer falls behind?
- What happens if a third-party API becomes slow or unavailable?
- Where can retries create duplicate operations?

Step 5: Explain Trade-offs

- Caching can reduce latency, but it introduces invalidation and stale-data problems.
- Replication can improve availability and read throughput, but it can create consistency lag.
- Sharding can increase writing capacity, but it makes queries, transactions, and operations harder.
- Asynchronous processing can absorb spikes, but users may not see results immediately.

5. Common Trade-offs to Discuss

Decision	Trade-off
Cache vs. Database	Cache improves speed and reduces DB load but adds stale data and invalidation complexity.
Replication vs. Consistency	More replicas can improve availability/read capacity, while synchronous coordination can increase latency.
Sharding vs. Simplicity	Sharding increases scale but introduces routing, rebalancing, and cross-shard query challenges.
Sync vs. Async	Synchronous calls give immediate results but couple services; asynchronous processing improves decoupling and spike handling.
Availability vs. Strong Consistency	During certain failures, distributed systems may need to choose which guarantee is more important for a given operation.
Cost vs. Performance	More replicas, larger instances, aggressive caching, and lower latency usually increase costs.

6. Failure Scenarios Interviewers Commonly Ask

- **Application server failure:** Load balancer removes unhealthy instance; other instances continue serving traffic.
- **Database failure:** Use replicas/failover where required; define what happens to write during failover.
- **Cache failure:** Fall back to the database carefully; prevent a cache stampede that can overload the DB.
- **Queue consumer failure:** Messages should remain retryable; use visibility timeout/retry policy and dead-letter handling where appropriate.
- **External API outage:** Use timeouts, retries with exponential backoff, circuit breakers, fallback behavior, and idempotency.
- **Traffic spike:** Autoscale stateless services, use caching, queues, rate limits, and load shedding where appropriate.
- **Network partition:** Define acceptable consistency and availability behavior; avoid indefinite blocking.

7. A Strong Interview Answer Structure

1. Clarify requirements and explicitly state assumptions.
2. Define scale: users, RPS, storage, peak traffic, and latency.
3. Identify the main APIs and important data entities.
4. Draw the simplest architecture that satisfies the requirements.
5. Explain the request/data flow end-to-end.
6. Find bottlenecks and introduce components only where they solve a specific problem.
7. Discuss database choice, indexing, partitioning, replication, and consistency.
8. Discuss caching, queues, retries, idempotency, and rate limiting where relevant.
9. Discuss availability, failure handling, security, observability, and deployment.
10. Close with trade-offs and explain what you would change if scale increased further.